Project 2 - Pokémon world in MongoDB

Group 8: Simone Cotardo, Srimal Fonseka, Arthur Morgan, Stipe Peran

1 Project scenario

Following the first project implemented in Neo4j, we decided to model the same Pokémon world in MongoDB. Instead of focusing on graph relationships such as `EVOLVES_TO` or `OWNS`, this second project focuses on storing and querying semistructured documents efficiently.

MongoDB's document model suits this domain well, as Pokémon, trainers, gyms and battles naturally contain nested attributes, arrays and flexible structures that do not require strong normalization.

The real data used in the project comes from five JSON files:

- `pokemon.json`
- `trainer.json`
- `gym.json`
- `type.json`
- `battles.json`

2 Methodology

2.1 Dataset and preprocessing

We reused the dataset previously collected for the Neo4j project. The main source was a JSON dataset containing Pokémon information (ID, name, stats, type, and evolution line). Data were cleaned and normalized using Python (Pandas), and exported as multiple collections in MongoDB.

The database was deployed using **MongoDB Compass** for visualization and the **mongo shell** for queries.

2.2 Collections and schema design

The document model was designed according to the *Schema-on-Read* philosophy introduced during the NoSQL lectures. Each document stores semistructured data in JSON format, allowing variable fields and embedded subdocuments.

As explained in the Project scenario introduction, we began with the original ER model and the previous Neo4J structure, then adapted and refined several attributes to better suit MongoDB's document-oriented design. For instance, we chose not to create a separate `Form` collection. Instead, we embedded that information directly within each Pokémon document, where it fits more naturally. As shown in Figure 2, the final result of this transformation can be clearly observed in the structure of the MongoDB database. The `mixed` field type indicates that a value may be either `null` or `string`, reflecting optional attributes within the dataset.

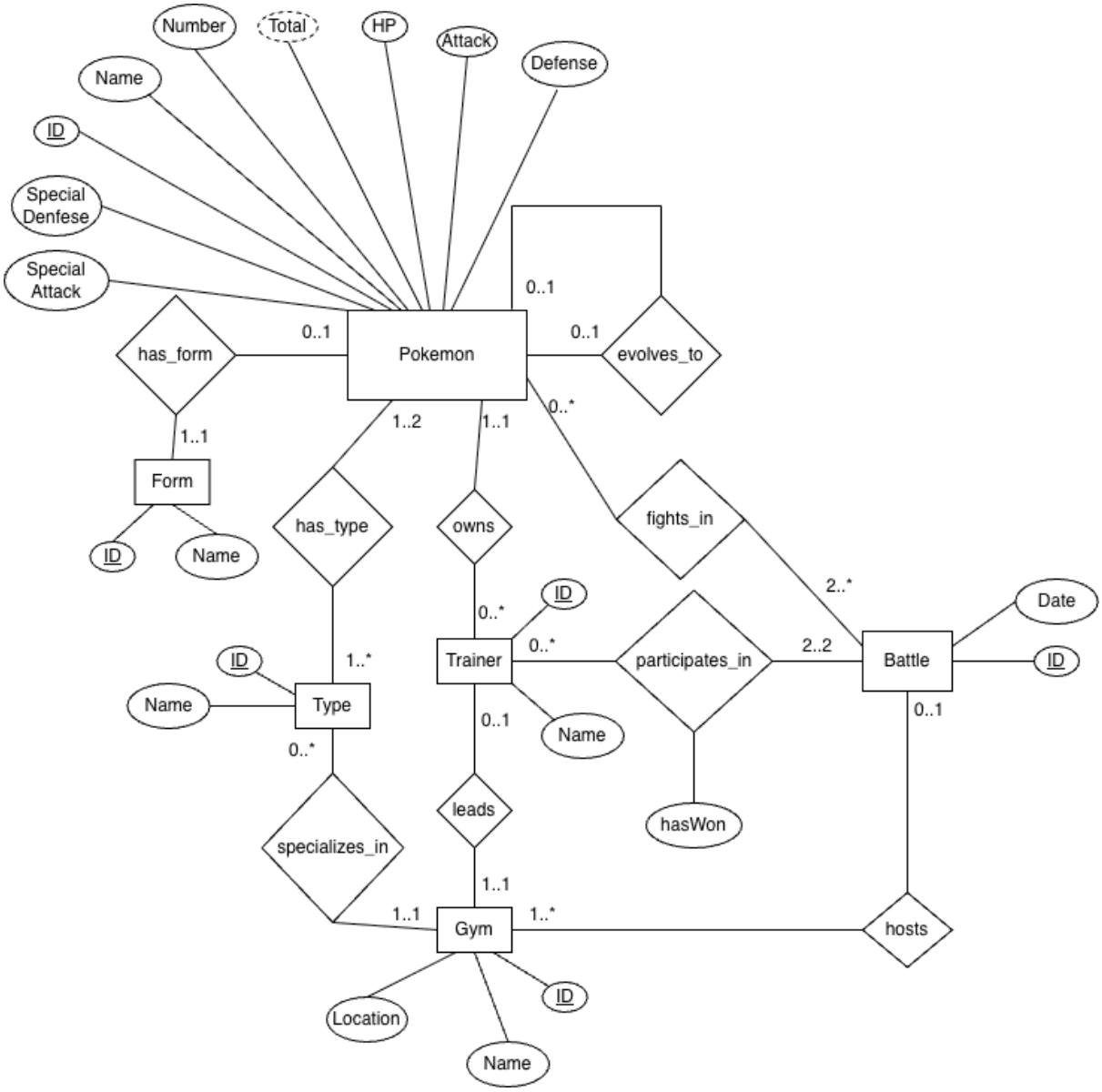

Figure 1: ER diagram

Collections

- **Pokemon**: canonical Pokémon entries with attributes, types, forms and evolutions.
- **Trainer**: documents describing each trainer, the Pokémon they own and which gym they lead.
- **Gym**: gym entities and their corresponding specialty types.
- **Type**: static collection containing the 18 Pokémon types.
- **Battle**: match history between trainers and Pokémon.

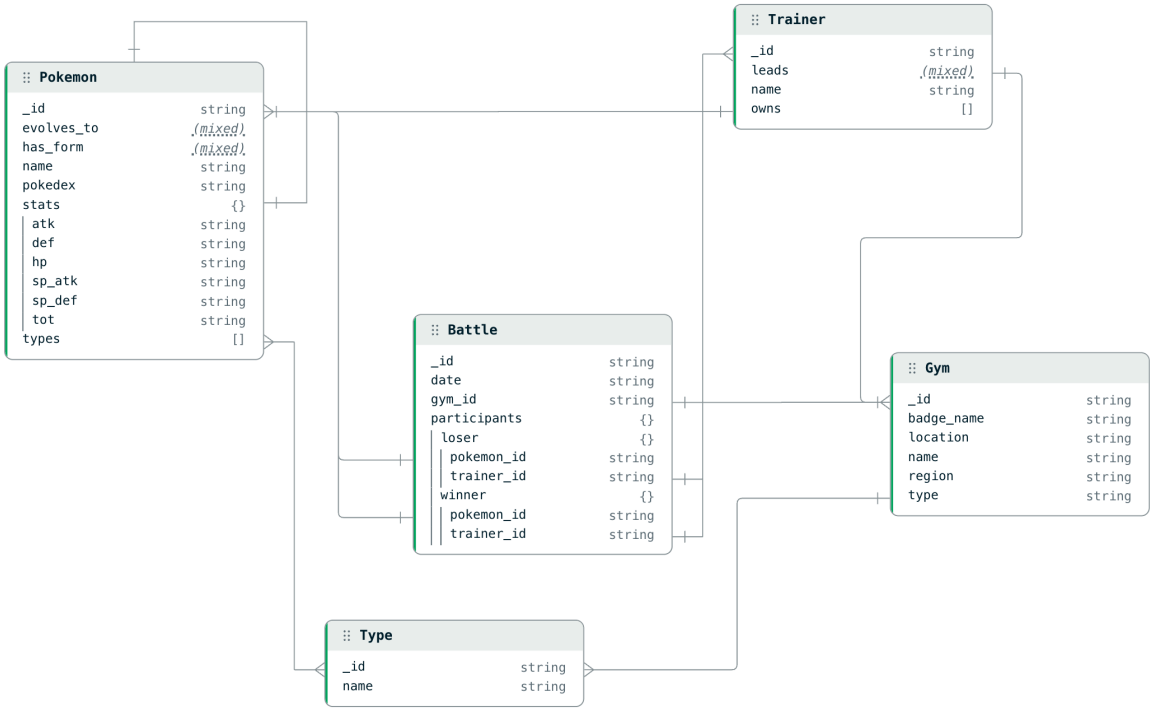

Figure 2: MongoDB structure

3 Results

3.1 Queries

Twelve representative queries were implemented to demonstrate real-world data retrieval, filtering, and aggregation. For each query, performance was measured by executing the command `.explain("executionStats")` and examining the value of `executionStats.executionTimeMillis` in the output, which reports the total execution time of the query.

3.1.1 For each trainer, their most successful Pokémon

The execution time for the following query was measured **68** ms.

```
db.Battle.aggregate([
  {
    $group: {
      _id: {
        trainer_id: "$participants.winner.trainer_id",
        pokemon_id: "$participants.winner.pokemon_id"
      },
      wins: { $sum: 1 }
    }
  },
  { $sort: { "_id.trainer_id": 1, wins: -1 } },
  {
    $group: {
      _id: "$_id.trainer_id",
      top: { $first: { pokemon_id: "$_id.pokemon_id", wins: "$wins"
          } }
    }
```

```
    },
    {
      $lookup: {
        from: "Trainer",
        localField: "_id",
        foreignField: "_id",
        as: "trainer"
      }
    },
    { $unwind: "$trainer" },
    {
      $lookup: {
        from: "Pokemon",
        localField: "top.pokemon_id",
        foreignField: "_id",
        as: "pokemon"
      }
    },
    { $unwind: "$pokemon" },
    {
      $project: {
        _id: 0,
        trainer: "$trainer.name",
        pokemon: "$pokemon.name",
        wins: "$top.wins"
      }
    },
    { $sort: { trainer: 1 } }
  ]);
```

3.1.2 For each trainer, the gym where they won the most battles

The execution time for the following query was measured **76** ms.

```
db.Battle.aggregate([
  {
    $group: {
      _id: {
        trainer_id: "$participants.winner.trainer_id",
        gym_id: "$gym_id"
      },
      wins: { $sum: 1 }
    }
  },
  { $sort: { "_id.trainer_id": 1, wins: -1 } },
  {
    $group: {
      _id: "$_id.trainer_id",
      top: { $first: { gym_id: "$_id.gym_id", wins: "$wins" } }
    }
  },
  {
    $lookup: {
      from: "Trainer",
      localField: "_id",
      foreignField: "_id",
      as: "trainer"
    }
```

```
    },
    { $unwind: "$trainer" },
    {
      $lookup: {
        from: "Gym",
        localField: "top.gym_id",
        foreignField: "_id",
        as: "gym"
      }
    },
    { $unwind: "$gym" },
    {
      $project: {
        _id: 0,
        trainer: "$trainer.name",
        gym: "$gym.name",
        wins: "$top.wins"
      }
    },
    { $sort: { trainer: 1 } }
  ]);
```

3.1.3 Most winning trainer

The execution time for the following query was measured **11** ms.

```
db.Battle.aggregate([
  {
    $group: {
      _id: "$participants.winner.trainer_id",
      wins: { $sum: 1 }
    }
  },
  { $sort: { wins: -1 } },
  { $limit: 1 },
  {
    $lookup: {
      from: "Trainer",
      localField: "_id",
      foreignField: "_id",
      as: "trainer"
    }
  },
  { $unwind: "$trainer" },
  {
    $project: {
      _id: 0,
      trainer: "$trainer.name",
      wins: 1
    }
  }
]);
```

3.1.4 Most winning pokemon

The execution time for the following query was measured **4** ms.

```
db.Battle.aggregate([
```

```
    {
      $group: {
        _id: "$participants.winner.pokemon_id",
        wins: { $sum: 1 }
      }
    },
    { $sort: { wins: -1 } },
    { $limit: 1 },
    {
      $lookup: {
        from: "Pokemon",
        localField: "_id",
        foreignField: "_id",
        as: "pokemon"
      }
    },
    { $unwind: "$pokemon" },
    {
      $project: {
        _id: 0,
        pokemon_name: "$pokemon.name",
        wins: 1
      }
    }
  ]);
```

3.1.5 Most winning trainer + most winning Pokémon

The execution time for the following query was measured **27** ms.

```
db.Battle.aggregate([
  {
    $group: {
      _id: "$participants.winner.trainer_id",
      wins: { $sum: 1 }
    }
  },
  { $sort: { wins: -1 } },
  { $limit: 1 },
  {
    $lookup: {
      from: "Trainer",
      localField: "_id",
      foreignField: "_id",
      as: "trainer"
    }
  },
  { $unwind: "$trainer" },
  {
    $project: {
      _id: 0,
      kind: "trainer",
      name: "$trainer.name",
      wins: 1
    }
  },
  {
    $unionWith: {
```

```
      coll: "Battle",
      pipeline: [
        {
          $group: {
            _id: "$participants.winner.pokemon_id",
            wins: { $sum: 1 }
          }
        },
        { $sort: { wins: -1 } },
        { $limit: 1 },
        {
          $lookup: {
            from: "Pokemon",
            localField: "_id",
            foreignField: "_id",
            as: "pokemon"
          }
        },
        { $unwind: "$pokemon" },
        {
          $project: {
            _id: 0,
            kind: "pokemon",
            name: "$pokemon.name",
            wins: 1
          }
        }
      ]
    }
  }
]);
```

3.1.6 Wins for each Pokémon and its evolutions (sum over evolution chain)

```
db.Pokemon.aggregate([
  {
    $lookup: {
      from: "Pokemon",
      localField: "_id",
      foreignField: "evolves_to",
      as: "prevForms"
    }
  },
  { $match: { prevForms: { $eq: [] } } },

  // Get full evolution line starting from the base
  {
    $graphLookup: {
      from: "Pokemon",
      startWith: "$_id",
      connectFromField: "evolves_to",
      connectToField: "_id",
      as: "line"
    }
  },

  // Build an array of all Pokemon IDs in the line: base +
```

```
    descendants
{
  $project: {
    rootPokemon: "$name",
    lineIds: {
      $concatArrays: [
        ["$_id"],
        {
          $map: {
            input: "$line",
            as: "p",
            in: "$$p._id"
          }
        }
      ]
    }
  }
},

// Count wins for any Pokemon in this line
{
  $lookup: {
    from: "Battle",
    let: { ids: "$lineIds" },
    pipeline: [
      {
        $match: {
          $expr: {
            $in: ["$participants.winner.pokemon_id", "$$ids"]
          }
        }
      },
      { $count: "wins_in_chain" }
    ],
    as: "winsAgg"
  }
},

// Normalize missing wins to 0
{
  $addFields: {
    wins_in_chain: {
      $ifNull: [
        { $arrayElemAt: ["$winsAgg.wins_in_chain", 0] },
        0
      ]
    }
  }
},

{
  $project: {
    _id: 0,
    rootPokemon: 1,
    wins_in_chain: 1
  }
},
{ $sort: { wins_in_chain: -1, rootPokemon: 1 } }
```

```
]);
```

3.1.7 Gym that hosted the highest number of battles

The execution time for the following query was measured **86** ms.

```
db.Battle.aggregate([
  {
    $group: {
      _id: "$gym_id",
      hosted: { $sum: 1 }
    }
  },
  { $sort: { hosted: -1 } },
  { $limit: 1 },
  {
    $lookup: {
      from: "Gym",
      localField: "_id",
      foreignField: "_id",
      as: "gym"
    }
  },
  { $unwind: "$gym" },
  {
    $project: {
      _id: 0,
      gym: "$gym.name",
      hosted: 1
    }
  }
]);
```

3.1.8 For each gym, total number of distinct Pokémon that fought there

The execution time for the following query was measured **25** ms.

```
db.Battle.aggregate([
  {
    $project: {
      gym_id: 1,
      pokemon_ids: [
        "$participants.winner.pokemon_id",
        "$participants.loser.pokemon_id"
      ]
    }
  },
  { $unwind: "$pokemon_ids" },
  {
    $group: {
      _id: { gym_id: "$gym_id", pokemon_id: "$pokemon_ids" }
    }
  },
  {
    $group: {
      _id: "$_id.gym_id",
      fighters: { $sum: 1 }
    }
```

```
    },
    {
      $lookup: {
        from: "Gym",
        localField: "_id",
        foreignField: "_id",
        as: "gym"
      }
    },
    { $unwind: "$gym" },
    {
      $project: {
        _id: 0,
        gym: "$gym.name",
        fighters: 1
      }
    },
    { $sort: { fighters: -1 } }
  ]);
```

3.1.9 Pokémon that fought in the most different gyms

The execution time for the following query was measured **29** ms.

```
db.Battle.aggregate([
  {
    $project: {
      gym_id: 1,
      winner_pokemon: "$participants.winner.pokemon_id",
      loser_pokemon: "$participants.loser.pokemon_id"
    }
  },
  {
    $project: {
      pairs: [
        { gym_id: "$gym_id", pokemon_id: "$winner_pokemon" },
        { gym_id: "$gym_id", pokemon_id: "$loser_pokemon" }
      ]
    }
  },
  { $unwind: "$pairs" },
  {
    $group: {
      _id: {
        pokemon_id: "$pairs.pokemon_id",
        gym_id: "$pairs.gym_id"
      }
    }
  },
  {
    $group: {
      _id: "$_id.pokemon_id",
      gymCount: { $sum: 1 }
    }
  },
  { $sort: { gymCount: -1 } },
  { $limit: 10 },
  {
```

```
      $lookup: {
        from: "Pokemon",
        localField: "_id",
        foreignField: "_id",
        as: "pokemon"
      }
    },
    { $unwind: "$pokemon" },
    {
      $project: {
        _id: 0,
        pokemon: "$pokemon.name",
        gymCount: 1
      }
    }
  ]);
```

3.1.10 Pokémon type with the best winning ratio

The execution time for the following query was measured **57** ms.

```
db.Battle.aggregate([
  // reshape battle into "fights" documents (pokemon_id + isWin)
  {
    $project: {
      winner_pokemon: "$participants.winner.pokemon_id",
      loser_pokemon: "$participants.loser.pokemon_id"
    }
  },
  {
    $project: {
      fights: [
        { pokemon_id: "$winner_pokemon", isWin: true },
        { pokemon_id: "$loser_pokemon", isWin: false }
      ]
    }
  },
  { $unwind: "$fights" },
  {
    $group: {
      _id: "$fights.pokemon_id",
      fights: { $sum: 1 },
      wins: {
        $sum: { $cond: ["$fights.isWin", 1, 0] }
      }
    }
  },
  // join Pokemon to get types
  {
    $lookup: {
      from: "Pokemon",
      localField: "_id",
      foreignField: "_id",
      as: "pokemon"
    }
  },
  { $unwind: "$pokemon" },
  { $unwind: "$pokemon.types" },
```

```
    {
      $group: {
        _id: "$pokemon.types",
        fights: { $sum: "$fights" },
        wins: { $sum: "$wins" }
      }
    },
    { $match: { fights: { $gt: 0 } } },
    {
      $project: {
        wins: 1,
        fights: 1,
        winRatio: { $divide: ["$wins", "$fights"] }
      }
    },
    { $sort: { winRatio: -1 } },
    { $limit: 5 },
    // attach type name
    {
      $lookup: {
        from: "Type",
        localField: "_id",
        foreignField: "_id",
        as: "typeInfo"
      }
    },
    { $unwind: "$typeInfo" },
    {
      $project: {
        _id: 0,
        type: "$typeInfo.name",
        winRatio: { $round: ["$winRatio", 3] },
        wins: 1,
        fights: 1
      }
    }
  ]);
```

3.1.11 Number of Pokémon that are at maximum evolution

The execution time for the following query was measured **2** ms.

```
db.Pokemon.aggregate([
  {
    $match: {
      $or: [
        { "evolution.next": null },
        { "evolution.next": { $exists: false } }
      ]
    }
  },
  { $count: "numMaxEvolution" }
]);
```

3.1.12 Most powerful Pokémon (by total)

The execution time for the following query was measured **3** ms.

```
db.Pokemon.aggregate([
  {
    $project: {
      _id: 0,
      name: 1,
      tot: "$stats.tot",
      form: "$has_form"
    }
  },
  { $sort: { tot: -1 } },
  { $limit: 10 }
]);
```

3.1.13 Most powerful Pokémon for each type

The execution time for the following query was measured **6** ms.

```
db.Pokemon.aggregate([
  { $unwind: "$types" },
  { $sort: { "stats.tot": -1 } },
  {
    $group: {
      _id: "$types",
      pokemon: { $first: "$name" },
      total: { $first: "$stats.tot" }
    }
  },
  {
    $lookup: {
      from: "Type",
      localField: "_id",
      foreignField: "_id",
      as: "typeInfo"
    }
  },
  { $unwind: "$typeInfo" },
  {
    $project: {
      _id: 0,
      type: "$typeInfo.name",
      pokemon: 1,
      total: 1
    }
  },
  { $sort: { type: 1 } }
]);
```

3.1.14 Group all Pokémon of type Water

The execution time for the following query was measured **3** ms.

```
db.Type.aggregate([
  { $match: { name: "Water" } },
  {
    $lookup: {
      from: "Pokemon",
      localField: "_id",
      foreignField: "types",
```

```
        as: "pokemon"
      }
    },
    { $unwind: "$pokemon" },
    {
      $project: {
        _id: 0,
        pokemon: "$pokemon.name"
      }
    },
    { $sort: { pokemon: 1 } }
  ]);
```

3.1.15 Highest improvement (total) from base to max evolution

```
db.Pokemon.aggregate([
  // Base evolutions: no previous form
  {
    $lookup: {
      from: "Pokemon",
      localField: "_id",
      foreignField: "evolves_to",
      as: "prevForms"
    }
  },
  { $match: { prevForms: { $eq: [] } } },

  // Get all evolutions in the line (descendants)
  {
    $graphLookup: {
      from: "Pokemon",
      startWith: "$_id",
      connectFromField: "evolves_to",
      connectToField: "_id",
      as: "line"
    }
  },

  // Convert totals to numbers
  {
    $addFields: {
      baseTotal: { $toInt: "$stats.tot" },
      lineTotals: {
        $map: {
          input: "$line",
          as: "p",
          in: { $toInt: "$$p.stats.tot" }
        }
      }
    }
  },

  // Max descendant total (or base itself if no descendants)
  {
    $addFields: {
      maxDescTotal: {
        $cond: [
```

```
            { $gt: [{ $size: "$lineTotals" }, 0] },
            { $max: "$lineTotals" },
            "$baseTotal"
          ]
        }
      }
    },

    // Improvement = maxDescTotal - baseTotal
    {
      $addFields: {
        improvement: { $subtract: ["$maxDescTotal", "$baseTotal"] }
      }
    },

    {
      $project: {
        _id: 0,
        basePokemon: "$name",
        baseTotal: 1,
        maxDescTotal: 1,
        improvement: 1
      }
    },
    { $sort: { improvement: -1, basePokemon: 1 } },
    { $limit: 20 }
  ]);
```

3.1.16 Gyms and their type specialization

The execution time for the following query was measured **5** ms.

```
db.Gym.aggregate([
  {
    $lookup: {
      from: "Type",
      localField: "type",
      foreignField: "_id",
      as: "typeInfo"
    }
  },
  { $unwind: "$typeInfo" },
  {
    $project: {
      _id: 0,
      gym: "$name",
      specializesIn: "$typeInfo.name"
    }
  },
  { $sort: { gym: 1, specializesIn: 1 } }
]);
```

3.2 Editing commands

Five additional editing commands were implemented to illustrate how MongoDB handles inserts, updates, and deletes. These operations mimic real-world changes such as new Pokémon discovery or stat correction. While MongoDB offers strong flexibility for modifying documents,

update operations that involve multiple "joins" tend to require more elaborate coordination across collections when compared to Neo4J's built-in graph traversal. To measure execution time, each command was wrapped between two `Date.now()` calls: one before the operation and one afterward.

3.2.1 Add a Pokémon

The execution time for the following query was measured **2** ms.

```
const newPokemon = {
  _id: "p9999",
  pokedex: "9999",
  name: "DataDesignosaur",
  evolves_to: null,
  has_form: null,
  stats: { hp: "99", atk: "99", def: "99", sp_atk: "99", sp_def: "
     99", tot: "499" },
  types: ["5", "12"]
};

db.Pokemon.insertOne(newPokemon);
const newPokemonId = newPokemon._id;
```

3.2.2 Move a Pokémon from one trainer to another

The execution time for the following query was measured **17** ms.

```
db.Trainer.updateOne({owns: "199"}, {$pull: {owns: "199"}})
db.Trainer.updateOne({_id: "1"}, {$push: {owns: "199"}})
```

3.2.3 Remove a "middle" evolution and connect base ev. to final ev.

The execution time for the following query was measured **10** ms.

```
const baseId = "1";  // Basic PokemonId

const base = db.Pokemon.findOne({ _id: baseId });
const middleId = base.evolves_to;
const middle = db.Pokemon.findOne({ _id: middleId });
const finalId = middle.evolves_to;

// Any Pokemon that evolved into middle should now evolve into
   final
db.Pokemon.updateMany(
  { evolves_to: middleId },
  { $set: { evolves_to: finalId } }
);

// Delete the middle evolution
db.Pokemon.deleteOne({ _id: middleId });
```

3.2.4 Remove the gym leader with the lowest win ratio and delete the trainer

The execution time for the following query was measured **167** ms.

```
// Only gym leaders
const worstLeader = db.Trainer.aggregate([
  {
```

```
    $match: {
      leads: { $exists: true, $ne: null }
    }
  },

  // Look up all battles where this trainer participated (winner OR
      loser)
  {
    $lookup: {
      from: "Battle",
      let: { tid: "$_id" },
      pipeline: [
        {
          $match: {
            $expr: {
              $or: [
                { $eq: ["$participants.winner.trainer_id", "$$tid"]
                    },
                { $eq: ["$participants.loser.trainer_id", "$$tid"]
                    }
              ]
            }
          }
        }
      ],
      as: "battles"
    }
  },

  // total battles and wins for this trainer
  {
    $addFields: {
      total: { $size: "$battles" },
      wins: {
        $size: {
          $filter: {
            input: "$battles",
            as: "b",
            cond: {
              $eq: ["$$b.participants.winner.trainer_id", "$_id"]
            }
          }
        }
      }
    }
  },

  // Ignore leaders with no battles
  {
    $match: {
      total: { $gt: 0 }
    }
  },

  // win ratio = wins / total
  {
    $addFields: {
      ratio: {
```

```
            $divide: ["$wins", "$total"]
          }
        }
      },

      // Worst first: lowest ratio, then fewest battles as tiebreaker
      {
        $sort: {
          ratio: 1,
          total: 1
        }
      },
      { $limit: 1 }
    ]).toArray()[0];

    db.Trainer.deleteOne({ _id: worstLeader._id });
```

3.2.5 Create the trainer Mario, give him the two strongest free Pokémon and assign him to an ownerless gym

The execution time for the following query was measured **1606** ms.

```
    const freeStrongest = db.Pokemon.aggregate([
      // Join with trainers that own this pokemon
      {
        $lookup: {
          from: "Trainer",
          localField: "_id",
          foreignField: "owns",
          as: "owners"
        }
      },
      // Keep only pokemon with no owners
      {
        $match: {
          owners: { $eq: [] }
        }
      },
      // Turn stats.tot (string) into a number for correct sorting
      {
        $addFields: {
          totalInt: { $toInt: "$stats.tot" }
        }
      },
      // Strongest first
      {
        $sort: { totalInt: -1 }
      },
      // Grab only the top 2
      { $limit: 2 }
    ]).toArray();

    const marioPokemonIds = freeStrongest.map(p => p._id);
    marioPokemonIds;

    // find an ownerless gym
    const freeGym = db.Gym.aggregate([
      {
```

```
      $lookup: {
        from: "Trainer",
        localField: "_id",
        foreignField: "leads",
        as: "leaders"
      }
    },
    {
      $match: {
        leaders: { $eq: [] }  // gyms with no leader
      }
    },
    { $limit: 1 }
]).toArray()[0];

const marioGymId = freeGym._id;
marioGymId;

// Create Mario
const marioId = "t9999";

const mario = {
  _id: marioId,
  name: "Mario",
  owns: marioPokemonIds,
  leads: marioGymId
};

db.Trainer.insertOne(mario);
```

4 Discussion

Compared to Neo4j, MongoDB excels in:

- **Data ingestion speed:** loading JSON documents was faster than importing CSVs and building relationships in Neo4j.
- **Flexibility:** the schema-less design model supports embedded structures and makes it easy to add or modify fields over time.
- **Horizontal scalability:** MongoDB scales out with sharding, as introduced in the lecture on data architectures.

However, Neo4j remains more efficient for relationship-heavy queries such as multi-hop evolutions, thanks to its index-free adjacency. In MongoDB, equivalent operations required more explicit aggregations and joining logic between collections.

5 Conclusion

This project successfully migrated the Pokémon world model from a graph to a document-oriented representation. We demonstrated how MongoDB can manage complex entities like Pokémon, trainers, and battles within flexible JSON structures. While Neo4j is superior for analyzing relationships, MongoDB provides better performance and simplicity for hierarchical, nested, and document-centric data.

Future work may include integrating the dataset into a web interface or building an analytics dashboard using MongoDB Atlas and Aggregation Pipelines.